Operators and Operands

- Arithmetic Operator
- Unary operators
- Relational and logical operators
- Assignment operators and
- Conditional operator

- The data items that operators act upon, are called *operands.*

Arithmetic Operators

- *There are five arithmetic operators in C. They are*

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Reminder after integer division

- *There is no exponentiation operator in C.*
- *There is a library function (POW) under math.h to carry out exponentiation*

Arithmetic Operators (cont.)

- The operands acted upon by arithmetic operators must represent numeric values.
- Thus, the operands can be integer quantities, floating-point quantities or characters
    - Character constant represent integer value

Arithmetic Operators (cont.)

- **The remainder operator** (%) requires that both operands be integers and the second operand be nonzero.
- Similarly, **the division operator (/)** requires that the second operand be nonzero.

```
4   int main()
5   {
6       int a, b;
7
8       a = 10;
9       b = 3;
10
```

a + b	→	13
a - b	→	7
a * b	→	30
a / b	→	3
a % b	→	1

Here, the decimal portion of the quotient will be dropped

```
4   int main()
5   {
6       float a, b;
7
8       a = 12.5;
9       b = 2.0
10
```

a + b	→	14.5
a - b	→	10.5
a * b	→	25.0
a / b	→	6.25
a % b	→	

Not Possible!!!

```
int main()
{
    char a, b;

    a = 'A';
    b = 'B'
```

a	→	65
a + b	→	131
a + 1	→	66
a + ‘A’	→	130
a + ‘1’	→	114

```
int a, b;

a = 11;
b = -3;
```

Follow basic rules of algebra

a + b	→	8
a - b	→	14
a * b	→	-33
a / b	→	-3
a % b	→	2

Ignore - !!!

Type Convention

- Operands that differ in type may undergo type conversion before the expression takes on its final value.
- In general, the final result will be expressed in the highest precision possible, consistent with the data types of the operands.
- Rules apply when neither operand is unsigned.

We will learn detail later

```
int i = 7;
float f = 5.5;
char c = 'w'
```

ASCII Value
w = 119
0 = 48

i + f	12.5	float (double)
i + c	126	integer
i + c – '0'	78	integer
(i + c) - (2 * f / 5)	123.8	float (double)

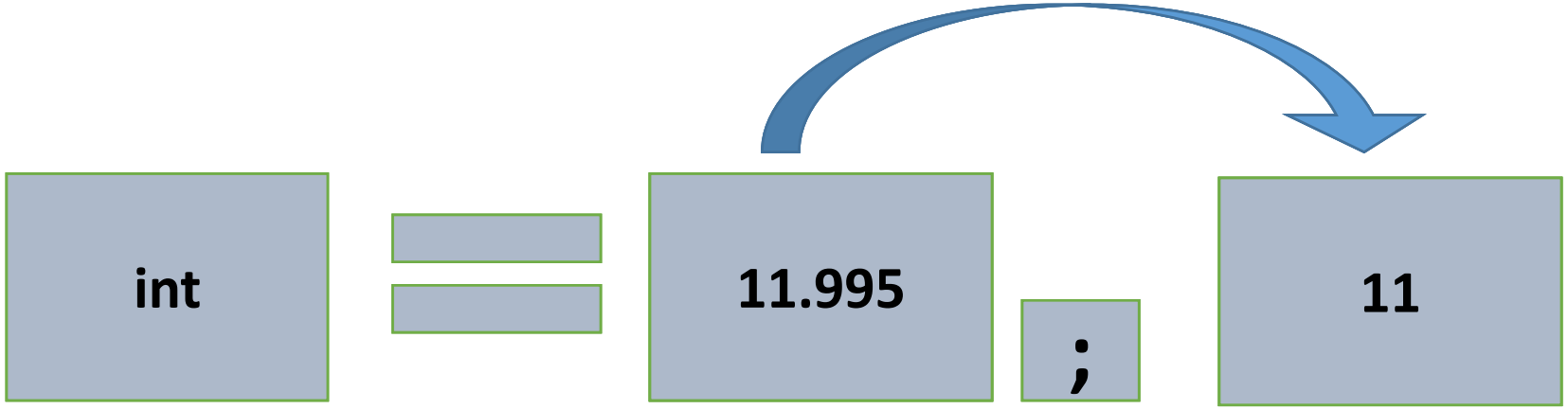
```c
int main()
{
    int a = 10;
    float f = 5.5, sum;

    sum = a + f;

    printf("%f",sum);

    return 0;
}
```

- [Type conversion]

Type Convention (Assignment)

- If the **two operands in assignment** expression **are of different data types.**
- The **value of right ha** ically be **converted to the type of the opera**
- The entire assignment expression will be then same data type.

We will learn detail later



Type Cast

- To transform the type of a variable temporarily.
- To do so, the expression must be preceded by the name of the desired data type, enclosed in parentheses

```
(data type) expression
```

```
int number;
(float) number;
```

Valid or Invalid?

```
6    int i, result;
7    float f;
8
```

```
i = 7;
f = 8.5;
```

```
result = (i + f) % 4;
```

- Type conversion 2

Valid or Invalid?

```
float num = 10.5;

num % 2;
```

```
float num = 10.5;

((int)num) % 2;
```

Operators and Operands

- Arithmetic Operator
- Unary operators
- Relational and logical operators
- Assignment operators and
- Conditional operator

Unary Operators

- A class of operators that **act upon a single operand** to produce a new value.
- Known as unary operators.

Unary Operators…

We will learn detail later

- Perhaps the most common unary operation is unary minus (-8)
- There are two other commonly used unary operators:
    - The increment operator, ++
    - The decrement operator, --
- sizeof - returns the size of its operand, in bytes.
- Cast - (type) expression

Edit View Search Project Build Debug wxSmith Tools Plugins Settings Help

main() : int

Build target: Debug

Workspace
Size of variables
Sources

main.c

```c
int main()
{
    int i;
    float x;
    double d;
    char c;

    printf ("integer: %d\n",sizeof i);
    printf ("float: %d\n",sizeof x);
    printf ("double: %d\n",sizeof d);
    printf ("character: %d\n",sizeof c);

    return 0;
}
```

"E:\C Code\Teach Yourself C\Chapter 01 - Fundamentals\Size of variables\bin\Debug\Size of variables.exe"

```
integer: 4
float: 4
double: 8
character: 1

Process returned 0 (0x0)   execution tim
Press any key to continue.
```

de\Teach Yourself C\Chapter 01 - Fundamentals\Size of variables\main.c | WINDOWS-1252 | Line 11, Column 29 | Insert | Read/Write | default

Relational Operators

- There are four relational operators in C. They are

Operator	*Meaning*
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

Equality Operators

- For equality

Operator	*Meaning*
==	equal to
!=	not equal to

Relational Operator

-

Operator	Meaning	Type
<	Less than	Relational
>	Greater than	Relational
<=	Less than or equal to	Relational
>=	Greater than or equal to	Relational
==	Equal to	Equality
!=	Not equal to	Equality

Relational Operators…

- These six operators are used to form logical expressions, which represent conditions that are either true or false.
- The resulting expressions will be of type integer

- *True is represented by the integer value 1*
- *False is represented by the value 0*

Examples of Conditions

Operator	Condition	Meaning
<=	*x* <= 0	*x* less than or equal to 0
<	`Power` < `MAX_POW`	`Power` less than `MAX_POW`
==	`letter` == ‘M’	`Letter` equal to ‘M’
!=	`num` != `x`	`num` not equal to `x`

Operator	Name	Example	Description
==	Equal operator	**A == Bi;**	True if **A** equals **B** else False.
!=	Not Equal operator	**A != B;**	True if **A** does not equal **B** else False.
<	Less Than operators	**A < B;**	True if **A** is less than **B** else False.
>	Greater Than operator	**A > B;**	True if **A** is greater than **B** else False.
<=	Less Than or Equal operator	**A <= B;**	True if **A** is less than or equals **B** else False.
>=	Greater Than or Equal operator	**A >= B;**	True if **A** is greater than or equals **B** else False.

True or False

i = 1
j = 2
k = 3

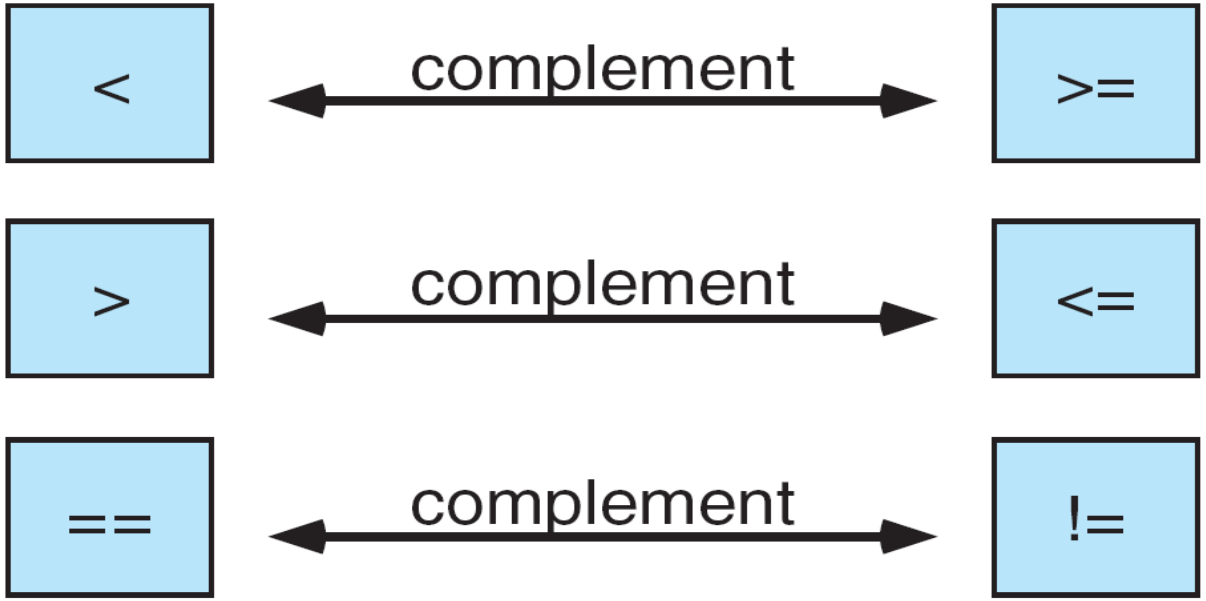
Expression	Result	Value
i < j	true	1
(1 + j) >= k	true	1
(j + k) > (i + 5)	false	0

True or False

i = 1
j = 2
k = 3

Expression	Result	Value
k != 3	false	0
j == 2	true	1
(j + k) >= (i + 5)	false	0

Type	Operator	Meaning	Precedence
Relational	< <= > >=	less than less than or equal greater than greater than or equal	10
Equality	== !=	equal not equal	9



Simplified Expression

Original Expression	Simplified Expression
`!(x < y)`	`x >= y`
`!(x > y)`	`x <= y`
`!(x != y)`	`x == y`
`!(x <= y)`	`x > y`
`!(x >= y)`	`x < y`
`!(x == y)`	`x != y`

Logical Operation

- There are three kinds of **logical operators**.

Operator	Name	Example
&&	AND operator	A && B;
||	OR operator	A || B;
!	NOT operator	!A;

লম্বা?	হাসিখুশি?	লম্বা এবং হাসিখুশি?
হ্যাঁ	হ্যাঁ	হ্যাঁ
হ্যাঁ	না	না
না	হ্যাঁ	না
না	না	না

লম্বা?	হাসিখুশি?	লম্বা অথবা হাসিখুশি?
হ্যাঁ	হ্যাঁ	হ্যাঁ
হ্যাঁ	না	হ্যাঁ
না	হ্যাঁ	হ্যাঁ
না	না	না

and

x	y	x&&y
false	false	false
false	true	false
true	false	false
true	true	true

or

x	y	xIIy
false	false	false
false	true	true
true	false	true
true	true	true

not

x	!x
false	true
true	false

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

AND

A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

OR

A	not A
0	1
1	0

NOT

True or False!

false && (anything)

false

true || (anything)

true

True or False

i = 7
f = 5.5
c = ‘w’ (119)

Expression	Result	Value
(i >= 6) && (c == 'w')	true	1
(i >= 6) || (c == 119)	true	1
(f < 11) && (i > 100)	false	0

Operator Precedence

An operator's **precedence** determines its order of evaluation.

Operator category	*Operators*	*Associativity*
unary operators	- ++ -- ! sizeof (*type*)	R → L
arithmetic multiply, divide and remainder	* / %	L → R
arithmetic add and subtract	+ -	L → R
relational operators	< <= > >=	L → R
equality operators	== !=	L → R
logical *and*	&&	L→ R
logical *or*	||	L → R

Operators and Operands

- Arithmetic Operator
- Unary operators
- Relational and logical operators
- Assignment operators and
- Conditional operator

Assignment operators

- Most common “=“
- = and == are not same!
- Other five are:
    - +=
    - -=
    - *=
    - /=
    - %=

Assignment operators….

- expression 1 += expression 2

is equivalent to

- expression 1 = expression 1 + expression 2

Example: `sum = sum + N;` → `sum+=N;`

Similarly, the assignment expression

- expression I -= expression 2

is equivalent to

- expression 1 = expression I - expression 2

Operator Precedence

Operator category	*Operators*	*Associativity*
unary operators	- ++ -- ! sizeof (*type*)	R → L
arithmetic multiply, divide and remainder	* / %	L → R
arithmetic add and subtract	+ -	L → R
relational operators	< <= > >=	L → R
equality operators	== !=	L → R
logical *and*	&&	L→ R
logical *or*	||	L → R
assignment operators	= += -= *= /= %=	R → L